

Module 15) Advance Python Programming

9. SQLite3 and PyMySQL (Database Connectors)

- **Introduction to SQLite3 and PyMySQL for database connectivity.**

1. SQLite3

- SQLite is a lightweight, serverless, self-contained, file-based relational database.
- It is embedded within the Python standard library via the sqlite3 module.
- Ideal for small to medium applications, prototyping, or when you want a simple database without installing a separate server.

Key Features:

- No separate server installation required.
- Stores entire database in a single file.
- Supports most SQL features.
- Good for local and embedded use.

Using SQLite3 in Python:

- Import the sqlite3 module.
- Connect to a database file (or create one if it doesn't exist).
- Execute SQL queries to create tables, insert, update, or fetch data.
- Commit changes and close the connection.

2. PyMySQL

- **PyMySQL** is a third-party Python library to connect to **MySQL** databases.
- MySQL is a popular, full-featured **client-server relational database management system (RDBMS)**.
- Requires a running MySQL server and appropriate credentials.

Key Features:

- Supports all standard MySQL features.
- Allows connecting to remote or local MySQL servers.
- Used for scalable, production-level applications.
- Supports transactions, stored procedures, and more.

Using PyMySQL in Python:

- Install via `pip install pymysql`.
 - Import PyMySQL and create a connection using host, user, password, and database name.
 - Use cursor objects to execute SQL commands.
 - Commit transactions and close connections properly.
-
- **Creating and executing SQL queries from Python using these connectors.**

1. Using SQLite3

Steps to create and execute SQL queries:

1. **Import** the `sqlite3` module.
2. **Connect** to the database file (creates if it doesn't exist).

3. **Create a cursor** object — it executes SQL commands.
4. Use the cursor to **execute SQL queries** (CREATE, INSERT, SELECT, etc.).
5. **Commit** changes (for modifying queries).
6. **Fetch results** for SELECT queries.
7. **Close** the connection

2. Using PyMySQL

Steps to create and execute SQL queries:

1. **Install and import** the pymysql module.
2. **Connect** to a MySQL server with credentials.
3. Create a **cursor** object.
4. **Execute SQL queries** using the cursor.
5. **Commit** for data-modifying queries.
6. **Fetch results** from SELECT queries.
7. **Close** cursor and connection.